🎮 从 Gemini 2 到 Gemini 335L：在 Jetson Orin NX 上复现深度相机体感游戏



> 一份面向复现、展示与二次开发的工程教程
>
> 原项目来源：深圳大学创新短课项目《基于 Gemini2 深度相机开发的体感游戏》
>
> 原作：沈墨、洪也淳、郑瑾慈
>
> 本文复现设备：NVIDIA Jetson Orin NX + Orbbec Gemini 335L
>
> 复现目标：让原有 Windows/Gemini2 项目在 Jetson Orin NX 上以 Orbbec 官方 SDK 方式读取 RGB + Depth，实现可运行、可调试、可展示的体感游戏 Demo。

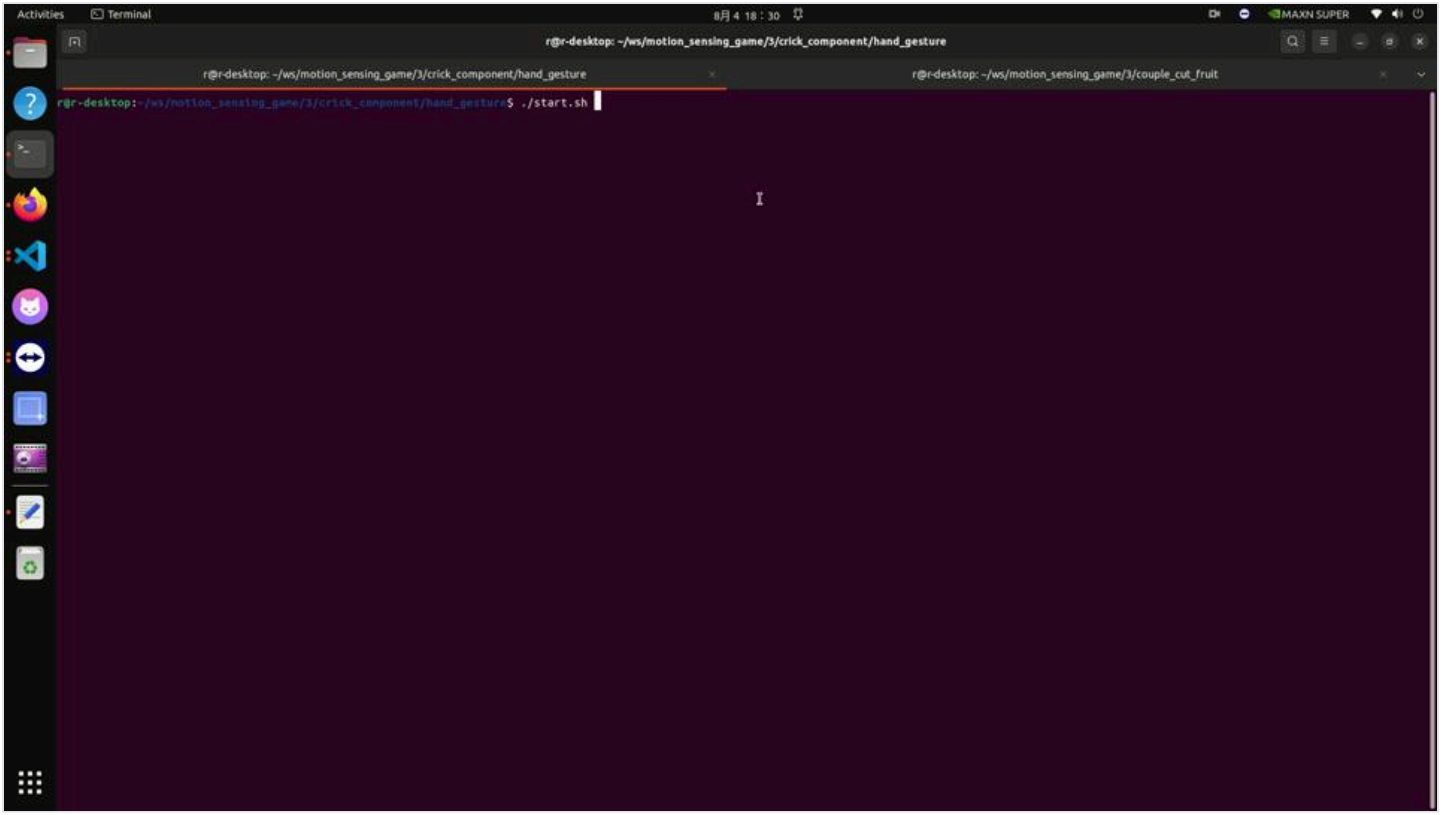
1. 项目背景与复现说明

本项目改编自深圳大学创新短课项目《基于 Gemini2 深度相机开发的体感游戏》。原项目围绕 Orbbec Gemini2 深度相机展开，使用 Python、OpenCV、MediaPipe、Pygame 等技术，实现了两

个体感交互游戏：

- **双人水果对抗**：两名玩家分别位于画面左右两侧，通过手掌侧边完成水果切割和炸弹躲避。
- **积木拼搭**：玩家通过握拳抓取、张手释放的方式移动虚拟积木，完成关卡目标图形。

本次复现并非简单复制运行，而是将项目从原来的 **Windows + Gemini2 + OpenCV OBSENSOR** 路线，迁移为更适合 Jetson 的 **Jetson Orin NX + Orbbec Gemini 335L + Orbbec SDK** 路线。



需要特别说明的是：本文使用的相机是 **Orbbec Gemini 335L**，不是原项目中的 Gemini2。Gemini 335L 同样属于 Orbbec 深度相机系列，复现时重点在于保证 RGB 彩色流、Depth 深度流、手部识别和游戏输入链路能够正确闭环。

2. 复现硬件与软件环境

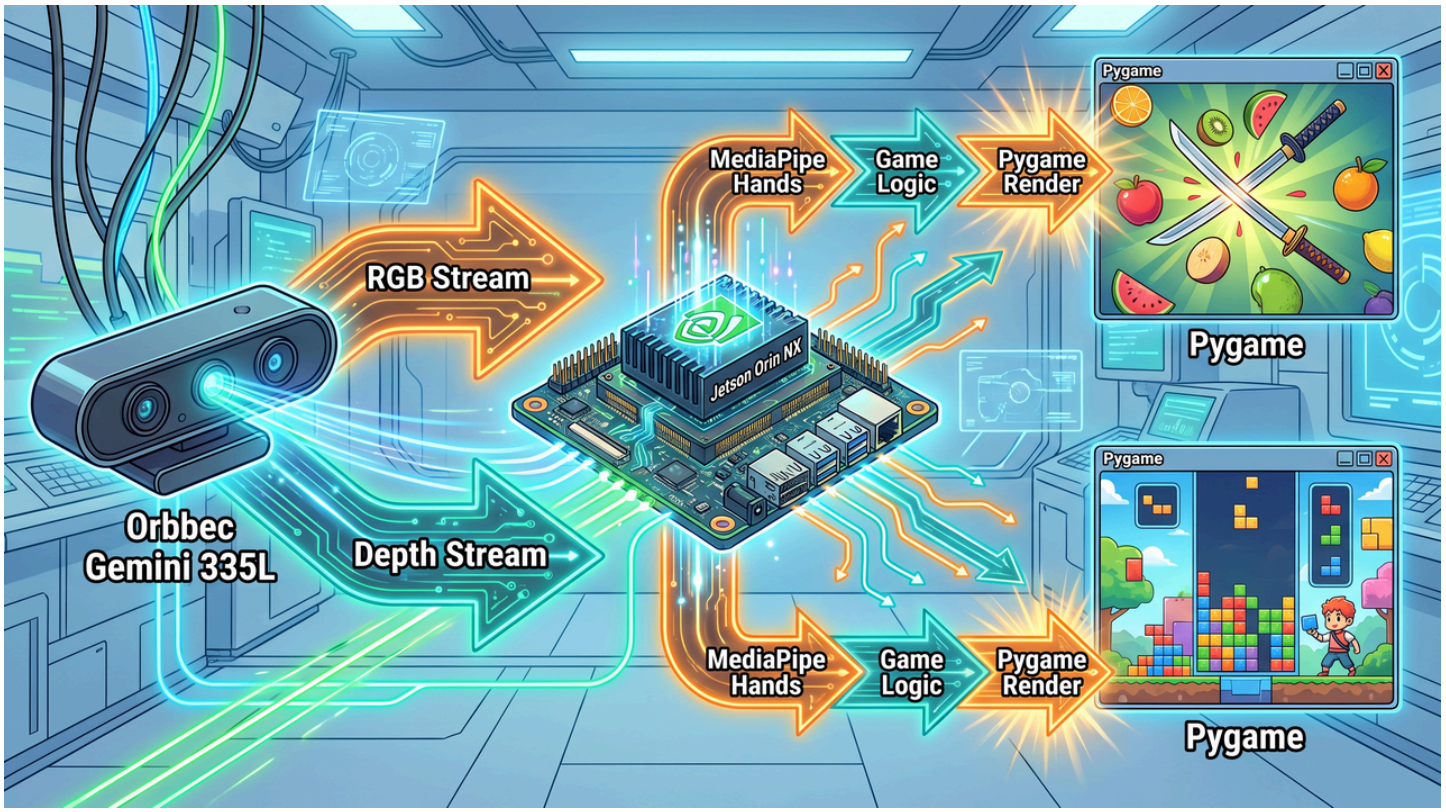
2.1 硬件环境

设备	说明

NVIDIA Jetson Orin NX	边缘 AI 计算设备，负责运行 Python 游戏与视觉识别
Orbbec Gemini 335L	本次复现使用的 RGB-D 深度相机
USB 3.0 数据线	建议使用质量稳定的数据线，避免丢帧或供电不稳
HDMI 显示器 / 触控屏	显示 Pygame 游戏窗口
键盘鼠标	调试、切换输入模式、退出程序

2.2 软件环境

软件	作用
Ubuntu / JetPack 系统	Jetson 官方 Linux 环境
Python 3	运行项目主程序
Orbbec SDK / pyorbbecsdk2	读取 Gemini 335L 彩色流和深度流
OpenCV	图像格式转换、镜像、缩放、摄像头 fallback
MediaPipe / cvzone	手部关键点识别
Pygame	游戏窗口、绘图、事件处理
NumPy	深度图、坐标、碰撞与速度计算



📌 参考资料：

- Orbbec Python SDK 安装文档：<https://orbbec.github.io/pyorbbecsdk/source/2_installation/install_the_package.html>

- Orbbec SDK v2 平台支持说明：<https://orbbec.github.io/OrbbecSDK_v2/>

- MediaPipe Python 文档：<https://github.com/google-ai-edge/mediapipe/blob/master/docs/getting_started/python.md>

- NVIDIA JetPack 安装说明：<https://docs.nvidia.com/jetson/jetpack/6.0DP/install-jetpack/index.html>

3. 项目目录说明

项目主要位于：

代码块

```
motion_sensing_game/
├── 2/
│   ├── 双人切水果_技术文档.pdf
```

```
4   │   ├── 积木拼搭_技术文档.pdf
5   │   └── 演示视频
6   ├── 3/
7   │   ├── couple_cut_fruit/
8   │   │   ├── main.py
9   │   │   ├── pose_input.py
10  │   │   ├── camera_backends.py
11  │   │   ├── requirements-jetson.txt
12  │   │   └── run_game_jetson.sh
13  │   └── crick_component/
14  │       └── hand_gesture/
15  │           ├── main.py
16  │           ├── camera.py
17  │           ├── game_logic.py
18  │           ├── objects.py
19  │           ├── ui.py
20  │           ├── requirements-jetson.txt
21  │           └── run_game_jetson.sh
```

本次为了在 Jetson Orin NX 上完整复现，重点修改了以下文件：

文件	修改目的
`couple_cut_fruit/camera_backends.py`	新增 Orbbec SDK RGBD 相机后端，返回彩色帧和深度帧
`couple_cut_fruit/main.py`	默认输入源改为 `orbbec`，自动模式优先使用 Orbbec SDK
`couple_cut_fruit/pose_input.py`	修正镜像画面下的深度采样坐标
`hand_gesture/camera.py`	使用 Orbbec SDK 读取 RGB + Depth，并按真实帧尺寸解码
`hand_gesture/main.py`	增加命令行参数，便于 Jetson 现场调试
`hand_gesture/ui.py`	增加 Linux 中文字体路径
`requirements-jetson.txt`	Jetson 专用依赖说明
`run_game_jetson.sh`	Jetson 一键启动脚本

4. Jetson 软件安装步骤

4.1 更新系统依赖

代码块

```
sudo apt update
sudo apt install -y \
  python3-pip \
  python3-venv \
  python3-dev \
  python3-opencv \
  fonts-noto-cjk \
  v4l-utils
```

说明：

- `python3-opencv` 优先使用 JetPack / Ubuntu 系统提供的 OpenCV，通常比自行安装 `opencv-python` 更稳定。
- `fonts-noto-cjk` 用于解决 Pygame 中文显示方框的问题。
- `v4l-utils` 用于检查 USB 摄像头设备，例如 `v4l2-ctl --list-devices`。

4.2 创建 Python 虚拟环境

建议使用虚拟环境，避免系统 Python 包被污染：

代码块

```
cd ~/motion_sensing_game/3/couple_cut_fruit
python3 -m venv .venv --system-site-packages
source .venv/bin/activate
python3 -m pip install --upgrade pip
```

这里使用 `--system-site-packages`，是为了让虚拟环境能直接访问系统安装的 `python3-opencv`。

4.3 安装 Orbbec Python SDK

根据 Orbbec 官方文档，Python 包名为：

```
代码块
1   python3 -m pip install --upgrade pyorbbecsdk2
```

⚠️ 注意：虽然 pip 包名是 `pyorbbecsdk2`，但代码中导入模块仍然是：

```
代码块
1   import pyorbbecsdk
```

安装后检查：

```
代码块
1   python3 -m pip show pyorbbecsdk2
2   python3 - <<'PY'
3   import pyorbbecsdk
4   print("pyorbbecsdk import ok")
5   PY
```

4.4 配置 Linux 设备权限

如果程序提示没有权限访问相机，或者必须 `sudo` 才能运行，需要安装 udev 规则。Orbbec 官方文档建议运行 SDK 自带的环境配置脚本。

可先找到 SDK 安装路径：

代码块

```
python3 - <<'PY'
import os
import pyorbbecsdk
print(os.path.dirname(pyorbbecsdk.__file__))
PY
```

然后按实际路径执行：

代码块

```
sudo python3 /path/to/site-packages/pyorbbecsdk/shared/setup_env.py
sudo udevadm control --reload-rules
sudo udevadm trigger
```

执行完成后建议重新插拔 Gemini 335L。

4.5 安装项目 Python 依赖

双人切水果：

代码块

```
cd ~/motion_sensing_game/3/couple_cut_fruit
source .venv/bin/activate
python3 -m pip install -r requirements-jetson.txt
```

积木拼搭：

```
代码块
1   cd ~/motion_sensing_game/3/crick_component/hand_gesture
2   source ../../couple_cut_fruit/.venv/bin/activate
3   python3 -m pip install -r requirements-jetson.txt
```

如果 `mediapipe` 在 Jetson aarch64 上安装失败，需要根据当前 JetPack、Python 版本寻找兼容 wheel，或参考 MediaPipe 官方文档从源码构建。这个项目的手部识别依赖 MediaPipe/cvzone，若 MediaPipe 不可用，游戏窗口可能能打开，但体感输入不会正常工作。

5. 运行项目



5.1 运行双人切水果

```
代码块
```

```
cd ~/motion_sensing_game/3/couple_cut_fruit
source .venv/bin/activate
chmod +x run_game_jetson.sh
./run_game_jetson.sh
```

等价命令：

代码块

```
python3 main.py --input orbbec --show-fps
```

如果需要全屏：

代码块

```
python3 main.py --input orbbec --show-fps --fullscreen
```

如果临时使用普通 UVC 摄像头：

代码块

```
python3 main.py --input webcam --camera-index 0
```

如果没有相机，只想测试游戏逻辑：

代码块

```
python3 main.py --input keyboard
```

5.2 运行积木拼搭

代码块

```
cd ~/motion_sensing_game/3/crick_component/hand_gesture
source ../../couple_cut_fruit/.venv/bin/activate
chmod +x run_game_jetson.sh
./run_game_jetson.sh
```

等价命令：

代码块

```
python3 main.py --input orbbec
```

普通摄像头 fallback：

代码块

```
python3 main.py --input webcam --camera-index 0
```

6. 关键技术实现说明

6.1 为什么 Jetson 上优先使用 Orbbec SDK？

原项目在 Windows 上优先使用 OpenCV 的 `CAP_OBSENSOR` 后端读取 Gemini2。这种方式代码较短，但依赖 OpenCV 是否编译了 OBSENSOR 支持。Jetson 上不同系统镜像、OpenCV 构建方式差异较大，因此更稳妥的方式是直接使用 Orbbec 官方 SDK。

修改后的双人切水果相机后端如下：

代码块

```
class OrbbecRgbdCamera:
    """Gemini2 / Orbbec SDK RGBD 后端。

    Jetson Orin NX 上优先使用 Orbbec 官方 SDK。它不依赖 OpenCV 是否
    编译了 OBSENSOR 后端，并且可以同时返回彩色图和深度图。
    """

    def start(self) -> None:
        from pyorbbecsdk import Config, OBSensorType, Pipeline

        config = Config()
        self.pipeline = Pipeline()
        color_profiles = self.pipeline.get_stream_profile_list(OBSensorType.COLOR_SENSOR)
        color_profile = color_profiles.get_default_video_stream_profile()
        config.enable_stream(color_profile)

        depth_profiles = self.pipeline.get_stream_profile_list(OBSensorType.DEPTH_SENSOR)
        depth_profile = depth_profiles.get_default_video_stream_profile()
        config.enable_stream(depth_profile)

        self.pipeline.start(config)
```

这段代码完成了三件事：

1. 创建 Orbbec SDK 的 `Pipeline`。
2. 启用彩色传感器 `COLOR_SENSOR`。
3. 启用深度传感器 `DEPTH_SENSOR`。

这样，后续每一帧都能同时拿到 RGB 图像和 Depth 深度图，为体感交互提供空间信息。

6.2 RGB 图像解码

Orbbec 相机输出的彩色帧可能是 RGB 原始数据，也可能是 MJPG/JPEG 压缩数据，因此代码需要兼容两种情况：

代码块

```
def _decode_color(self, color_frame) -> Optional[np.ndarray]:
    width = color_frame.get_width()
    height = color_frame.get_height()
    data = np.frombuffer(color_frame.get_data(), dtype=np.uint8)

    if data.size >= width * height * 3:
        rgb = data[: width * height * 3].reshape((height, width, 3))
        return cv2.cvtColor(rgb, cv2.COLOR_RGB2BGR)

    decoded = cv2.imdecode(data, cv2.IMREAD_COLOR)
    if decoded is not None:
        return decoded
    return None
```

这里统一返回 OpenCV 常用的 BGR 格式，方便后续使用 `cv2.flip`、`cv2.resize`、`cv2.cvtColor` 等操作。

6.3 深度图解码与单位处理

深度帧一般以 `uint16` 形式存储。不同设备或 SDK 配置下，深度单位可能需要乘以 scale，代码中做了统一处理：

代码块

```
def _decode_depth(self, depth_frame) -> Optional[np.ndarray]:
    width = depth_frame.get_width()
    height = depth_frame.get_height()
    data = np.frombuffer(depth_frame.get_data(), dtype=np.uint16)

    depth = data[: width * height].reshape((height, width)).astype(np.float32)
    get_depth_scale = getattr(depth_frame, "get_depth_scale", None)
    if callable(get_depth_scale):
        scale = float(get_depth_scale())
        if scale > 0:
```

```
            depth *= scale
    return depth
```

这样做的目的，是尽量把深度值统一为毫米级距离，后续可以直接用于：

- 输出玩家手部距离；
- 过滤背景误触；
- 判断玩家是否处于有效交互范围；
- 进一步开发前景分割或空间玩法。

6.4 手掌刀输入：从手部关键点到切割线



双人切水果中，项目没有使用完整人体骨架，而是使用手腕和小拇指指尖之间的线段作为“手掌刀”：

代码块

```
wrist_lm = hand_lm.landmark[self.mp_hands.HandLandmark.WRIST]
pinky_lm = hand_lm.landmark[self.mp_hands.HandLandmark.PINKY_TIP]

```

```
4   wrist = self._scale_point((wrist_lm.x * self.process_width, wrist_lm.y *
    self.process_height))
5   pinky = self._scale_point((pinky_lm.x * self.process_width, pinky_lm.y *
    self.process_height))
6   blade = self._make_blade(wrist, pinky, hand_index, process_dt,
    depth_for_sampling)
```

这种设计比较自然：玩家伸出手掌侧边，从水果上扫过，即可触发切割。相比使用整条手臂，手掌侧边更短、更精确，也更容易做碰撞判断。

6.5 镜像画面与深度采样对齐

体感游戏通常会把相机画面水平镜像，使玩家看到的动作像照镜子一样自然。问题是：如果只镜像 RGB 图像，而不镜像深度图，手部关键点和深度坐标就会左右错位。

因此代码中同步处理：

代码块

```
1   frame = cv2.flip(bgr_frame, 1)
2   depth_for_sampling = cv2.flip(depth_frame, 1) if depth_frame is not None else
    None
```

这是一处非常关键的细节。否则终端可能能打印深度值，但它采样到的并不是手掌所在位置，而是画面另一侧的深度。

6.6 玩家左右分区

双人切水果按照手掌刀中心点判断玩家归属：

代码块

```
center = ((wrist[0] + pinky[0]) / 2.0, (wrist[1] + pinky[1]) / 2.0)
pid = 1 if center[0] < self.width / 2 else 2
```

左半屏归属玩家 1，右半屏归属玩家 2。水果和炸弹生成时也会带有 `owner_side`，避免玩家跨屏切掉对方区域的目标。

6.7 积木拼搭中的“深度锁手”

积木拼搭只需要追踪一只主要操作手。为了减少旁边其他人的手进入画面导致误控，项目使用掌心位置和深度值进行锁定：

代码块

```
if self._locked_depth and candidate.depth_mm:
    depth_gap = abs(candidate.depth_mm - self._locked_depth)
    if depth_gap > 450 and self._locked_palm is not None:
        self._missing_frames += 1
        return hand_data
```

这段逻辑的含义是：如果当前候选手和已经锁定的手在深度上相差太大，就不立即切换目标。这样可以降低多人围观场景中的误识别概率。

7. 复现验证清单

建议按以下顺序验证，不要一上来直接运行完整游戏。

7.1 检查相机是否被系统识别

代码块

```
lsusb
v4l2-ctl --list-devices
```

如果看不到 Orbbec 设备：

- 检查 USB 线是否支持数据传输；
- 尽量使用 USB 3.0 接口；
- 重新插拔相机；
- 确认相机没有被 Orbbec Viewer 或其他程序占用。

7.2 检查 Orbbec SDK 是否可导入

代码块

```
python3 - <<'PY'
import pyorbbecsdk
print("pyorbbecsdk ok")
PY
```

7.3 检查 OpenCV 是否可导入

代码块

```
python3 - <<'PY'
import cv2
print(cv2.__version__)
PY
```

7.4 检查 MediaPipe 是否可导入

代码块

```
python3 - <<'PY'
import mediapipe as mp
print(mp.__version__)
PY
```

如果这一步失败，后续手势识别大概率无法正常工作，应优先解决。

7.5 运行游戏并观察深度输出

双人切水果正常运行后，终端应周期性输出类似内容：

代码块

```
[Depth] P1: 836 mm | P2: 1042 mm
[Depth] P1: -- | P2: 1018 mm
```

其中：

- `P1` 表示左半屏玩家；
- `P2` 表示右半屏玩家；
- 单位为毫米；
- `--` 表示当前没有识别到该玩家手掌刀，或采样区域没有有效深度。

8. 常见问题与避坑点

8.1`pip install mediapipe` 失败

这是 Jetson 上最容易遇到的问题之一。MediaPipe 官方文档中提到，aarch64 Linux 系统可能需要从源码构建或使用适配 wheel。建议：

1. 先确认 Python 版本，例如：

```
python3 --version
```

2. 优先尝试：

```
python3 -m pip install mediapipe
```

3. 如果失败，再根据 JetPack 和 Python 版本寻找兼容 wheel，或参考官方源码构建流程。

8.2 相机无权限或必须 sudo 才能运行

通常是 udev 规则未配置。请运行 Orbbec SDK 的环境配置脚本，并重新插拔相机。

8.3 有 RGB 画面，但没有深度输出

可能原因：

- 深度流没有成功启用；
- Gemini 335L 当前 USB 连接不稳定；
- SDK profile 不兼容；
- 相机被其他程序占用；
- 程序退回了 `webcam` 模式。

建议观察窗口右下角的输入源标识：

- `orbbec:rgbd`：RGB + Depth 正常；
- `orbbec:color`：只有彩色流；
- `webcam:0`：当前使用普通摄像头；
- `keyboard`：相机不可用，已退回键盘模拟。

8.4 中文显示为方框

安装中文字体：

```
代码块
1   sudo apt install -y fonts-noto-cjk
```

如果仍然异常，可检查字体路径：

```
代码块
1   ls /usr/share/fonts/opentype/noto/
2   ls /usr/share/fonts/truetype/noto/
```

8.5 游戏画面卡顿

可尝试：

- 降低 MediaPipe 处理分辨率：

  ```
  代码块
  1   python3 main.py --input orbbec --pose-width 360 --show-fps
  ```

- 降低后台负载；
- 使用 Jetson 性能模式；
- 确认相机接在 USB 3.0；
- 避免同时打开 Orbbec Viewer。

8.6 深度值跳动明显

可能是手部边缘存在深度空洞。项目中已经使用 7x7 中位数采样来降低噪声：

代码块

```
patch = np.asarray(depth[y1:y2, x1:x2], dtype=np.float32).reshape(-1)
patch = patch[np.isfinite(patch) & (patch > 0)]
return float(np.median(patch))
```

如果仍然跳动，可考虑：

- 增大采样窗口；
- 限制有效距离范围；
- 对深度值做滑动平均；
- 让玩家手部距离相机保持在稳定范围内。

9. 推荐展示流程

为了让观众更容易理解项目，建议展示时按以下顺序讲解：

1. 介绍项目来源：深圳大学创新短课项目。
2. 展示硬件：Jetson Orin NX + Orbbec Gemini 335L。
3. 打开双人切水果，说明 RGB 图像用于手部识别，Depth 用于距离感知。
4. 观察终端 `[Depth]` 输出，让观众看到深度相机不是普通摄像头。
5. 展示积木拼搭，说明握拳/张手手势如何映射到游戏操作。
6. 最后介绍后续可扩展方向：前景分割、空间交互、多人身份管理、儿童益智关卡等。

🖼️ 技术讲解配图生成提示词

代码块

```
动漫线条风格，颜色艳丽，画面中一只手被 MediaPipe 标出 21 个关键点，手腕到小拇指指尖之间有一条发光切割线，旁边显示深度值 850mm，背景有水果和游戏 UI，科技教学插图，清晰线稿，16:9
```

10. 后续优化方向

当前复现已经完成了 RGB + Depth 数据链路和游戏交互闭环，后续可以继续优化：

- **有效距离过滤**：只允许 0.5m 到 2.5m 范围内的手参与交互。
- **深度前景分割**：利用深度图区分玩家和背景，减少复杂背景误触。
- **相机标定界面**：根据场地自动调整左右分区、有效高度和交互范围。
- **多人身份管理**：结合深度和位置持续追踪不同玩家。
- **性能统计面板**：显示 FPS、手部识别耗时、相机读取耗时、丢帧率。
- **展示包装**：加入音效、教程页、排行榜和项目说明页。

11. 总结

本次复现的核心工作，是把原本更偏 Windows/Gemini2/OpenCV OBSENSOR 的项目，迁移到 Jetson Orin NX 和 Orbbec Gemini 335L 上，以 Orbbec 官方 SDK 作为主要相机输入方式。

从技术链路看，项目已经形成了完整闭环：

代码块

```
Gemini 335L RGB + Depth
        ↓
Orbbec SDK / pyorbbecsdk2
        ↓
OpenCV 图像处理
        ↓
MediaPipe 手部关键点识别
```

```
          ↓
手掌刀 / 握拳张手输入
          ↓
Pygame 游戏逻辑与渲染
```

这个项目的价值不只在于“跑起来一个小游戏”，更在于它清楚展示了深度相机在体感交互中的作用：RGB 图像负责识别动作，Depth 深度图负责理解空间距离。二者结合后，才有机会进一步实现更稳定、更自然、更适合真实展示场景的人机交互体验。🌟